

Chapter 6

Context-Aware Recommender Systems

Gediminas Adomavicius and Alexander Tuzhilin

6.1 Introduction and Motivation

Many existing approaches to recommender systems focus on recommending the most relevant items to individual users and do not take into consideration any contextual information, such as time, place, and the company of other people (e.g., for watching movies or dining out). In other words, traditionally recommender systems deal with applications having only two types of entities, users and items, and do not put them into a context when providing recommendations.

However, in many applications, such as recommending a vacation package, personalized content on a Web site, or a movie, it may not be sufficient to consider only users and items—it is also important to incorporate the *contextual information* into the recommendation process in order to recommend items to users under certain *circumstances*. For example, using the temporal context, a travel recommender system would provide a vacation recommendation in the winter that can be very different from the one in the summer. Similarly, in the case of personalized content delivery on a Web site, it is important to determine what content needs to be delivered (recommended) to a customer and when. More specifically, on weekdays a user might prefer to read world news when she logs on in the morning and the stock market report in the evening, and on weekends to read movie reviews and do shopping.

G. Adomavicius (✉)

Department of Information and Decision Sciences, University of Minnesota,
321 19th Avenue South, Minneapolis, MN 55455, USA
e-mail: gedas@umn.edu

A. Tuzhilin

Department of Information, Operations and Management Sciences, Stern School of Business,
New York University, 44 West 4th Street, Rm 8-85, New York, NY 10012, USA
e-mail: atuzhili@stern.nyu.edu

These observations are consistent with the findings in behavioral research on consumer decision making in marketing that have established that decision making, rather than being invariant, is contingent on the context of decision making. Therefore, accurate prediction of consumer preferences undoubtedly depends upon the degree to which the recommender system has incorporated the relevant contextual information into a recommendation method.

Over the past 10–15 years, context-aware recommendation capabilities have been developed by academic researchers and applied in a variety of different application settings, including: movie recommenders [6], restaurant recommenders [68], travel recommenders and tourist guides [19, 35, 65, 73, 78, 90], general music recommenders [51, 72, 76], specialized music recommenders (e.g., for places of interest [27], in-car music [18], or music while reading [32]), mobile information search [39], news recommenders [60], shopping assistants [80], mobile advertising [29], mobile portals [85], mobile app recommenders [53], and many others. In particular, *mobile* recommender systems constitute an important special case of context-aware recommenders, where context is often defined by location and time, and there exists a large body of literature dedicated specifically to mobile recommender systems (e.g., see [19, 49, 88] for a few representative examples). In this chapter we focus on the issues related to the general area of context-aware recommender systems and, therefore, we do not provide a separate in-depth review of mobile recommender systems. Readers interested in a systematic coverage of mobile recommender systems are referred to Chap. 14 as well as two recent reviews of the field [54, 77].

Similarly, companies have also started incorporating some contextual information into their recommendation engines for recommending music (e.g., www.last.fm, musicoverly.com, and www.sourcetone.com) and movies (e.g., www.moviepilot.de and www.filmtipset.se). For example, when choosing which songs to play for a given user, some interactive radio stations ask the users to specify their current mood by selecting it from a list of well-established types of moods, such as “positive”, “energetic”, “calm”, “dark”, etc. The chosen mood is then used by the system as contextual information to recommend only the type of music that best fits the selected mood. As another example, the movie streaming and rental company Netflix knows the location of its customers and uses the locational contextual variables, such as city or zip code as well as time, to provide context-specific recommendations of movies. Similarly, mobile recommender systems (e.g., those deployed on smartphones) provide more relevant recommendations to its customers when they take into account such important contextual information as the GPS-based location and time. As Reed Hastings, the CEO of Netflix, pointed out in the following Youtube video [1] (see the video at 44:40 min), Netflix reportedly can improve the performance of its recommendation algorithms up to 3 % when taking into account such contextual information as the time of the day or user’s location. This observation by Hastings was echoed by the participants at the industrial panel held during the CARS workshop at the RecSys’12 conference (<http://cars-workshop.org/cars-12/program>), where managers from LinkedIn, Netflix,

EchoNest, and Telefonica reiterated the importance of contextual information and described how their recommendation engines utilize contextual information in their businesses.

In this chapter we review the topic of *context-aware recommender systems* (CARS). In particular, we discuss the notion of context and how it can be modeled in recommender systems. We also review the major approaches to modeling contextual information in recommender systems and discuss three main algorithmic paradigms for incorporating contextual information into rating-based recommender systems—contextual pre-filtering, post-filtering, and modeling. We also survey recent work on context-aware recommender systems and discuss important and promising directions for future research.

The rest of the chapter is organized as follows. Section 6.2 discusses the general notion of context as well as the general approaches to how it can be modeled in recommender systems. Section 6.3 presents three main algorithmic paradigms for incorporating contextual information into the rating-based recommender systems within the representational framework of modeling contextual information. Finally, some additional discussion and opportunities for future work are presented in Sect. 6.4.

6.2 Context in Recommender Systems

Before discussing the role and opportunities of contextual information in recommender systems, in Sect. 6.2.1 we start by presenting the general notion of context. Then, starting with Sect. 6.2.2, we focus on recommender systems and explain how context is specified and modeled there.

6.2.1 What is Context?

Context is a multifaceted concept that has been studied across different research disciplines, including computer science (primarily in artificial intelligence and ubiquitous computing), cognitive science, linguistics, philosophy, psychology, and organizational sciences. In fact, an entire conference—CONTEXT¹—is dedicated exclusively to studying this topic and incorporating it into various other branches of science, including medicine, law, and business. Since context is a multidisciplinary concept, each discipline tends to take its own idiosyncratic view that is somewhat different from other disciplines and is more specific than the standard generic dictionary definition of context as “conditions or circumstances

¹See, for example, <http://www.polytech.univ-savoie.fr/index.php?id=context-13-home>, <http://context-11.teco.edu>, or <http://context-07.ruc.dk>, for recent instances.

which affect some thing” [87]. Therefore, there exist many definitions of context across various disciplines and even within specific subfields of these disciplines. Bazire and Brézillon [25] present and examine 150 different definitions of context from different fields. This is not surprising, given the complexity and the multifaceted nature of the concept. As Bazire and Brézillon [25] observe:

... it is difficult to find a relevant definition satisfying in any discipline. Is context a frame for a given object? Is it the set of elements that have any influence on the object? Is it possible to define context a priori or just state the effects a posteriori? Is it something static or dynamic? Some approaches emerge now in Artificial Intelligence [...]. In Psychology, we generally study a person doing a task in a given situation. Which context is relevant for our study? The context of the person? The context of the task? The context of the interaction? The context of the situation? When does a context begin and where does it stop? What are the real relationships between context and cognition?

To bring some “order” to this diversity of views on what context is, Dourish [42] introduces taxonomy of contexts, according to which contexts can be classified into the representational and the interactional views. In the *representational* view, context is defined with a predefined set of observable attributes, the structure (or schema, using database terminology) of which does not change significantly over time. In other words, the representational view assumes that the contextual attributes are identifiable and known a priori and, hence, can be captured and used within the context-aware applications. In contrast, the *interactional* view assumes that the user behavior is induced by an underlying context, but that the context itself is not necessarily observable. Furthermore, Dourish [42] assumes that different types of actions may give rise to and call for different types of relevant contexts, thus assuming a *bidirectional* relationship between activities and underlying contexts: contexts influence activities and also different activities giving rise to different contexts.

In this chapter we focus on what context is *in recommender systems* and try to adapt this multifaceted concept to the specific and idiosyncratic domain of RSeS. Furthermore, we will also revisit and enhance the prior definitions of context used in recommender systems, including those provided in [5, 10]. We start with the standard and popular representational approach to modeling contextual information in Sect. 6.2.2, and explore and describe alternative approaches in Sect. 6.2.3.

6.2.2 Representational Approach to Modeling Contextual Information in Recommender Systems

Recommender systems emerged as an independent research area in the mid-1990s, when researchers and practitioners started focusing on recommendation problems that explicitly rely on the notion of ratings as a way to capture user preferences for different items. For example, in case of a movie recommender system, John Doe may assign a rating of 7 (out of 10) for the movie “Gladiator,” i.e., set $R_{movie}(\text{John_Doe}, \text{Gladiator}) = 7$. The rating-based recommender systems typically

start with the specification of the initial set of ratings that is either explicitly provided by the users or is implicitly inferred by the system. Once these initial ratings are specified, a recommender system tries to estimate the rating function R

$$R : User \times Item \rightarrow Rating$$

for the (user, item) pairs that have not been rated yet by the users. Here *Rating* is a totally ordered set (e.g., non-negative integers or real numbers within a certain range), and *User* and *Item* are the domains of users and items respectively. Once the function R is estimated for the whole $User \times Item$ space, a recommender system can recommend the highest-rated items for each user, possibly also taking into account item novelty, diversity, or other considerations of recommendation quality [84]. We call such systems *traditional* or *two-dimensional* (2D) since they consider only the *User* and *Item* dimensions in the recommendation process.

In other words, in its most common formulation, the traditional rating-based recommendation problem is reduced to the problem of estimating ratings for the items that have not been seen by a user. This estimation is usually based on the ratings given by the users to other items, and possibly on some other information, such as user demographics and item characteristics. Note that this traditional approach to RSes does not take into the consideration the *contextual information*, such as time, location and the company of other people.

The rating-based *representational approach to context-aware recommender systems* assumes that the contextual information is known and defined by a set of contextual attributes and that these contextual attributes affect ratings. In other words, the ratings are modeled in the rating-based representational approach to CARS as the function of not only items and users, but also of the contextual attributes, i.e., as

$$R : User \times Item \times Context \rightarrow Rating,$$

where *User* and *Item* are the domains of users and items respectively, *Rating* is the domain of ratings, and *Context* specifies the known contextual information associated with the application. To illustrate these concepts, consider the following example.

Example 6.1. Consider the application for recommending movies to users, where users and movies are described as relations having the following attributes:

- **Movie:** the set of all the movies that can be recommended; it is defined as Movie(MovieID, Title, Length, ReleaseYear, Director, Genre).
- **User:** the people to whom movies are recommended; it is defined as User(UserID, Name, Address, Age, Gender, Profession).

Further, the contextual information consists of the following three types that are also defined as relations having the following attributes:

- Theater: the movie theaters showing the movies; it is defined as Theater (TheaterID, Name, Address, Capacity, City, State, Country).
- Time: the time when the movie can be or has been seen; it is defined as Time(Date, DayOfWeek, TimeOfWeek, Month, Quarter, Year). Here, attribute DayOfWeek has values Mon, Tue, Wed, Thu, Fri, Sat, Sun, and attribute TimeOfWeek has values “Weekday” and “Weekend”.
- Companion: represents a person or a group of persons with whom one can see a movie. It is defined as Companion(companionType), where attribute companionType has values “alone”, “friends”, “girlfriend/boyfriend”, “family”, “co-workers”, and “others”.

Then the rating assigned to a movie by a person also depends on where and how the movie has been seen, with whom, and at what time. For example, the type of movie to recommend to college student Jane Doe can differ significantly depending on whether she is planning to see it on a Saturday night with her boyfriend vs. on a weekday with her parents.

As we can see from this example and other cases, the contextual information Context can be of different *types*, each type defining a certain aspect of context, such as time, location (e.g., Theater), companion (e.g., for seeing a movie), purpose of a purchase, etc. Further, each contextual type can have a complicated structure reflecting complex nature of the contextual information. Although this complexity of contextual information can take many different forms, one popular defining characteristic is the *hierarchical* structure of contextual information that can be represented as trees, as is done in most of the context-aware recommender and profiling systems, including [6, 69]. For instance, the three contexts from Example 6.1 can have the following hierarchies associated with them: *Theater*: TheaterID $\rightarrow$ City $\rightarrow$ State $\rightarrow$ Country; *Time*: Date $\rightarrow$ DayOfWeek $\rightarrow$ TimeOfWeek, Date $\rightarrow$ Month $\rightarrow$ Quarter $\rightarrow$ Year.²

This *representational* view to CARS assumes that the context is defined with a predefined set of observable attributes, the structure of which does not change significantly over time, and constitutes the most popular approach to incorporating context in RSes. More specifically, many context-aware approaches follow Palmisano et al. [69], and also Adomavicius et al. [6] to some extent, and define the contextual information with a *set of contextual dimensions* K , each contextual dimension K in K being defined by a set of q attributes $K = (K^1, \dots, K^q)$ having a hierarchical structure and capturing a particular type of context, such as *Time* or *CommunicatingDevice*. The values taken by attribute K^q define *finer* (more *granular*) levels, while K^1 values define *coarser* (less granular) levels of

²For the sake of completeness, we would like to point out that not only the contextual dimensions, but also the traditional User and Item dimensions can have their attributes form hierarchical relationships. For example, the main two dimensions from Example 6.1 can have the following hierarchies associated with them: *Movie*: MovieID $\rightarrow$ Genre; *User*: UserID $\rightarrow$ Age, UserID $\rightarrow$ Gender, UserID $\rightarrow$ Profession.

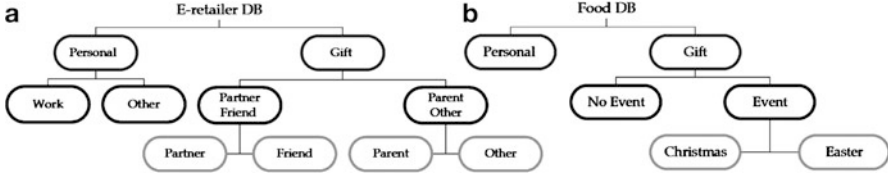


Fig. 6.1 Contextual information hierarchical structure: (a) e-retailer dataset, (b) food dataset [69]

contextual knowledge. For example, Fig. 6.1a presents a four-level hierarchy for the contextual attribute K specifying the intent of a purchasing transaction in an e-retailer application. While the root (coarsest level) of the hierarchy for K defines purchases in all possible contexts, the next level is defined by attribute $K^1 = \{Personal, Gift\}$, which labels each customer purchase either as a personal purchase or as a gift. At the next, finer level of the hierarchy, “Personal” value of attribute K^1 is further split into a more detailed personal context: personal purchase made for the work-related or other purposes. Similarly, the *Gift* value for K^1 can be split into a gift for a partner or a friend and a gift for parents or others. Thus, the K^2 level is $K^2 = \{PersonalWork, PersonalOther, GiftPartner/Friend, GiftParent/Other\}$. Finally, attribute K^2 can be split into further levels of hierarchy, as shown in Fig. 6.1a.³

Contextual information was also defined in [6] as follows. In addition to the classical *User* and *Item* dimensions, additional contextual dimensions, such as *Time*, *Location*, etc., were also introduced using the OLAP-based⁴ *multidimensional data (MD) model* widely used in the data warehousing applications in databases [37, 55]. Mathematically, this model can be defined with an n -dimensional *tensor*. Formally, let $D_1, D_2, \dots, D_n$ be dimensions, two of these dimensions being *User* and *Item*, and the rest being contextual. Each dimension D_i is a subset of a Cartesian product of some attributes (or fields) A_{ij} , ($j = 1, \dots, k_i$), i.e., $D_i \subseteq A_{i1} \times A_{i2} \times \dots \times A_{ik_i}$, where each attribute defines a domain (or a set) of values. Moreover, one or several attributes form a *key*, i.e., they uniquely define the rest of the attributes [75]. In some cases, a dimension can be defined by a single attribute, and $k_i = 1$ in such cases. For example, consider the three-dimensional recommendation space $User \times Item \times Time$, where the *User* dimension is defined as $User \subseteq UName \times Address \times Income \times Age$ and consists of a set of users having certain names, addresses, incomes, and being of a certain age. Similarly, the *Item* dimension is defined as $Item \subseteq IName \times Type \times Price$ and consists of a set of items defined by their names, types and the price. Finally, the *Time* dimension can be defined as $Time \subseteq Year \times Month \times Day$ and

³For simplicity and illustration purposes, this figure uses only two-way splits. Obviously, three-way, four-way and, more generally, multi-way splits are also allowed.

⁴OLAP stands for OnLine Analytical Processing, which represents a popular approach to manipulation and analysis of data stored in multi-dimensional cube structures and which is widely used for decision support.

consists of a list of days from the starting to the ending date (e.g. from January 1, 2003 to December 31, 2003).

Given dimensions $D_1, D_2, \dots, D_n$, we define the recommendation space for these dimensions as a Cartesian product $S = D_1 \times D_2 \times \dots \times D_n$. Moreover, let *Rating* be a rating domain representing the ordered set of all possible rating values. Then the *rating function* is defined over the space $D_1 \times \dots \times D_n$ as

$$R : D_1 \times \dots \times D_n \rightarrow \text{Rating}.$$

For instance, continuing the *User* $\times$ *Item* $\times$ *Time* example considered above, we can define a rating function R on the recommendation space *User* $\times$ *Item* $\times$ *Time* specifying how much user $u \in \text{User}$ liked item $i \in \text{Item}$ at time $t \in \text{Time}$, $R(u, i, t)$.

Visually, ratings $R(d_1, \dots, d_n)$ on the recommendation space $S = D_1 \times D_2 \times \dots \times D_n$ can be stored in a multidimensional cube, such as the one shown in Fig. 6.2. For example, the cube in Fig. 6.2 stores ratings $R(u, i, t)$ for the recommendation space *User* $\times$ *Item* $\times$ *Time*, where the three tables define the sets of users, items, and times associated with *User*, *Item*, and *Time* dimensions respectively. For example, rating $R(101, 7, 1) = 6$ in Fig. 6.2 means that for the user with User ID 101 and the item with Item ID 7, rating 6 was specified during the weekday.

The rating function R introduced above is usually defined as a partial function, where the initial set of ratings is known. Then, as usual in the ratings-based

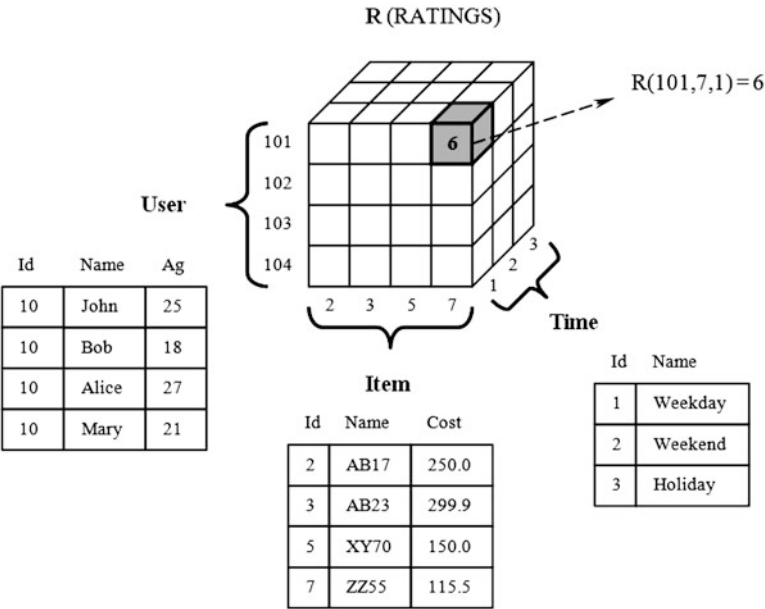


Fig. 6.2 Multidimensional model for the *User* $\times$ *Item* $\times$ *Time* recommendation space

recommender systems, the goal is to estimate the unknown ratings, i.e., make the rating function R total.

The main difference between the multidimensional (MD) contextual model described above and the previously described contextual model lies in that contextual information in the MD model is defined using classical OLAP hierarchies, whereas the contextual information in the previous case is defined with more general hierarchical taxonomies, which can be represented as trees (both balanced and unbalanced), directed acyclic graphs (DAGs), or various other types of taxonomies. Further, the ratings in the MD model are stored in the multidimensional cubes, whereas the ratings in the other contextual model are stored in more general hierarchical structures.

In this section we have provided an overview of the representational approach to modeling contextual information in CARS. The vast majority of CARS research employ this approach, and we will dedicate Sect. 6.3 of this chapter to the discussion of various recommendation techniques that utilize the contextual information using this representational approach in order to provide better recommendations. However, we first discuss several other approaches to modeling contextual information in recommender systems (i.e., besides the representational approach) in Sect. 6.2.3.

6.2.3 Major Approaches to Modeling Contextual Information in Recommender Systems

As mentioned earlier, many existing CARS approaches assume the existence of certain contextual factors (sometimes called contextual dimensions, variables or attributes), such as time, location, and the purchasing purpose, that identify the context in which recommendations are provided. As discussed in Sect. 6.2.2, each contextual factor can be defined by (a) its structure (e.g., defined using trees or OLAP hierarchies) and (b) the values that the contextual variables take. For example, the structure of the Time factor can be defined in terms of Years, Months, Days and Hours. Furthermore, each variable in this structure, Years, Months, Days and Hours, takes the standard set of values, such as the standard 12 months in a year, the standard number of days in a month, hours in a day, etc.

A broader classification of major approaches to modeling contextual information, i.e., classification that goes beyond the standard assumption of the explicit availability of predefined contextual factors with stable (unchanging) structure, is based on the following two aspects of contextual factors [5]: (1) what a recommender system may know about these contextual factors, and (2) how contextual factors change over time.

The first aspect presumes that a recommender system can have different types of knowledge about the contextual factors. This may include knowledge of the list of relevant factors, their structure, and their values. Depending on what is exactly known about the factors (that is, what is observed and what is not), we can classify this knowledge of a recommender system about the contextual factors

into the following three categories: fully observable, partially observable, and unobservable [5].

- *Fully observable*: The contextual factors relevant to the application, as well as their structure and their values at the time when recommendations are made, are known *explicitly*. For example, in case of recommending a purchase of a certain product, such as a shirt, the recommender system may know that only the Time, PurchasingPurpose, and ShoppingCompanion factors matter in this application. Further, the recommender system may know the relevant structure of all these three contextual factors, such as having categories of weekday, weekend, and holiday for Time. Further, the recommender system may also know the values of the contextual factors at the recommendation time (for example, when this purchase is made, with whom, and for whom).
- *Partially observable*: Only some of the information about the contextual factors, as described above, is known explicitly. For example, the recommender system may know all the contextual factors, such as Time, PurchasingPurpose, and ShoppingCompanion, but not their structure. Note that there can possibly be different levels of “partial observability.” In this chapter, we do not differentiate between them and group various cases of partially observable knowledge into this general category. As another example, mobile recommender systems may know the temporal and geo-spatial contextual factors (e.g., from the system clock and the GPS sensor of the mobile phone), but all other contextual factors may remain unknown.
- *Unobservable*: No information about contextual factors is explicitly available to the recommender system, and it makes recommendations by utilizing only the *latent* knowledge of context in an implicit manner. For example, the recommender system may build a latent predictive model, such as a hierarchical linear or hidden Markov model, to estimate unknown ratings, where unobservable context is modeled using latent variables.

The second aspect of contextual factors is whether and how their structure and their importance changes over time. The settings where the contextual factors are stable over time are classified as static, whereas the factors changing over time are classified as dynamic [5].

- *Static*: The relevant contextual factors and their structure remains the same (stable) over time. For example, in case of recommending a purchase of a certain product, such as a shirt, we can include the contextual factors of Time, PurchasingPurpose, ShoppingCompanion and only these factors during the entire lifespan of the purchasing recommendation application. Furthermore, we assume that the structure of the PurchasingPurpose factor does not change over time: the set of purposes will remain the same throughout the lifespan of the application. The same is applicable to the ShoppingCompanion factor when the same class of shopping companions remains throughout the application.
- *Dynamic*: This is the case when the contextual factors change in some way. For example, the recommender system (or the system designer) may realize over

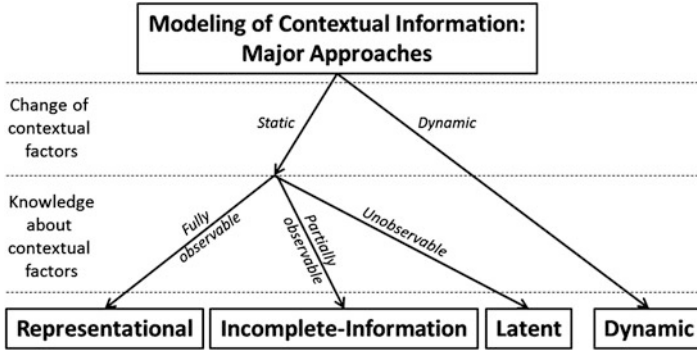


Fig. 6.3 Major approaches to modeling contextual information in recommender systems

time that the ShoppingCompanion factor is no longer relevant for purchasing recommendations and may decide to drop it. Furthermore, the structure of some of the contextual factors can change over time (for example, new categories can be added to the PurchasingPurpose contextual factor over time).

Directly combining the two aspects (i.e., the extent of the recommender system’s knowledge about contextual factors and their change over time) could provide 6 (i.e., 3×2) possible approaches to modeling contextual information. However, modeling of dynamic contexts represents an already complex task, for which the differentiation between observable and unobservable factors may not be easily formalizable. Therefore, we treat all the approaches to settings with dynamic contextual information as one unified category. In contrast, the modeling approaches for the static context settings are indeed classified further based on the three major cases according to what is known about the contextual factors. This results in four major, distinct approaches to modeling contextual information in recommender systems, as indicated in Fig. 6.3, and we describe them in the remainder of this section.

Representational Approach (Context: Static, Fully Observable) As discussed earlier, this approach corresponds to the *representational* view of context [42], which assumes that all the contextual information in a given application can be modeled with a predefined finite set of observable attributes, where each attribute has a well-defined structure and the structure does not change significantly over time (i.e., is static). The vast majority of the prior work in context-aware recommender systems has focused on this approach and, therefore, we will focus largely on this method in the remainder of this chapter.

Incomplete-Information Approach (Context: Static, Partially Observable) This approach explicitly assumes that only some partial information is known about the contextual factors, which are assumed to have static structure. As mentioned above, an example of the latter case can occur in mobile applications, when the temporal and geo-spatial contextual information is known to the mobile device, but

other important contextual factors (e.g., user's trip purpose, companions, mood) may remain unknown. Another example of such approach is presented in Palmisano et al. [69], where the contextual information is defined using a Bayesian Network (BN). In this BN, the observed context corresponds to the external layers of the network and the unobserved latent one to the middle layers of the BN. Further, some of the unobserved parameters of the BN were learned from the data using standard machine-learning methods.

Latent Approach (Context: Static, Unobservable) This approach represents recommendation settings with stable (i.e., static), yet directly unobservable contextual factors. For example, such contextual factors may include the mood of the user (happy, sad, etc.) or the intent of purchase (for yourself, for work, gift, etc.), modeling which could provide recommendation performance improvements. Because the contextual factor structure is stable, it can be modeled with latent variables, and the unobserved contextual information can be learned using machine-learning methods, such as matrix factorization [58], probabilistic latent semantic analysis (PLSA), or hierarchical linear models (HLMs). However, precisely because latent information is not directly observable, it may be difficult to differentiate contextual latent modeling from general-purpose latent modeling approaches in recommender systems (e.g., a variety of general-purpose matrix factorization approaches that are currently popular in recommender systems literature). One possible point of differentiation is that all the latent variables could potentially be associated with the item, user, and/or contextual characteristics. Therefore, the contextual latent variables are those latent characteristics that do not pertain to the user or the item. Although this is an intuitive distinction conceptually, it is hard to operationalize it in practice, and the development of such differentiation methods constitutes an interesting topic of future research and could lead to more nuanced recommendation models and, as a result, potential performance benefits.

Dynamic Approach (Context: Dynamic, Various Observability) This approach represents recommendation settings where the structure (as well as the pertinence) of the contextual information can change over time, e.g., based on passive observations or explicit user feedback. This approach is related to the *interactional* view of context [42], and there are some studies that take the direct interactional approach to modeling contextual recommendations, such as [14] which models context through a short-term memory (STM) interactional approach borrowed from psychology. As another example, Mahmood et al. [65] present a recommender system that adapts the dialogue to the interaction context. This is modeled by a set of dynamic contextual factors representing, for instance, whether the user provided certain information or acted on the recommendations (for example, put an item into the shopping cart). In a dynamic way, step by step, the system adapts the interaction considering a selection of these factors depending on the state of the interaction. In addition, Moling et al. [66] propose a recommendation approach based on a sequential decision making perspective, which allows to blend changing (i.e., less stable) implicit short-term/contextual preferences with more stable long-term user preferences when producing radio channel recommendation. As yet another

example, Hariri et al. [45] use a topic-modeling approach to map user's interaction sequence to a sequence of latent topics (representing different contexts) which capture trends in user's current interests. This allows the recommender system to monitor changes in users' interests and dynamically adapt to these changes. As yet another example, consider conversational recommender systems. In standard conversational systems, user feedback is used to iteratively refine the user profile (or the initial user query) resulting in more appropriate recommendations. In a context-aware conversational system, the user feedback may also be used to iteratively modify contextual factors and not just user profiles. For example, in the course of conversation with a user, a restaurant recommender may determine that the user is on a romantic date. This observation, in turn, may result in filtering out restaurants that tend to be noisy or without an adequate wine selection. The iterative refinement of context in conversational recommender systems will introduce a new set of research questions that will require further investigation.

6.2.4 Obtaining Contextual Information

As mentioned earlier, context-aware recommender systems research has been focusing on the representational approach, which assumes that the context is defined with a *predefined* set of contextual attributes, the structure of which does not change over time. The implication of this assumption is that contextual information needs to be identified and acquired before actual recommendations are made. The contextual information can be obtained in a number of ways, including:

- *Explicitly*, i.e., by directly approaching relevant people and other sources of contextual information and explicitly gathering this information either by asking direct questions or eliciting this information through other means. For example, a website may obtain contextual information by asking a person to fill out a web form or to answer some specific questions before providing access to certain web pages. Similarly, a smartphone app may obtain time, location, and motion data from the phone's clock, GPS sensor, and accelerometer, respectively.
- *Implicitly* from the data or the environment, such as a change in location of the user detected by a mobile telephone company. Alternatively, temporal contextual information can be implicitly obtained from the timestamp of a transaction. Nothing needs to be done in these cases in terms of interacting with the user or other sources of contextual information—the source of the implicit contextual information is accessed directly and the data is extracted from it.
- *Inferring* the context using statistical or data mining methods. In other words, contextual information can be “hidden” in the data in some latent form but can still be *implicitly* used to better estimate the unknown ratings. For example, the household identity of a person flipping the TV channels (husband, wife, son, daughter, etc.) may not be explicitly known to a cable TV company; but it can be modeled as a latent variable using various machine learning methods by observing the TV programs watched and the channels visited. This information can then be used to estimate how much this household would like a particular

TV program. It was shown in [69] that this deployment of latent variables, such as intent of purchasing a product (e.g., for yourself vs. as a gift, work-related vs. pleasure, etc.), whose true values were unknown but that were explicitly modeled as a part of a Bayesian Network (BN), indeed improved the predictive performance of the BN classifier. A similar approach of using latent variables is presented in [14]. As another example of inferring contextual information, consider online reviews, such as provided on Yelp, Amazon, and other popular websites. These reviews contain plenty of contextual information describing specific purchasing or consumption experiences, such as restaurant visits. For example, the user may indicate in a review that she went to the restaurant for dinner with her boyfriend to celebrate his birthday. Bauman and Tuzhilin [24] present a method of analyzing such online reviews and extracting contextual information from them. This method is also evaluated on a sample of Yelp reviews, and it is shown that most of the contextual information relevant to that application is extracted using this method. Still another approach toward inferring contextual information, albeit for non-RS related problems, was proposed in [56] where temporal contexts were discovered in web-sessions by decomposing these sessions into non-overlapping segments, each segment relating to one specific context. These contexts were subsequently identified using certain optimization and clustering methods.

It is important to note that, if the acquisition process of the contextual information is done explicitly or even implicitly, it should be conducted as a part of the overall data collection process. This further implies that the decisions of which contextual information should be relevant and collected for an application should be done at the application design stage and well in advance of the time when actual recommendations are provided.

Naturally, not all available contextual factors might be relevant or useful for recommendation purposes. Consider, for example, a book recommender system. Many types of contextual data could potentially be obtained by such a system from book buyers, including: (a) purpose of buying the book (possible options: for work, for leisure, etc.); (b) planned reading time (weekday, weekend, etc.); (c) planned reading place (at home, at school, on a plane, etc.); (d) the value of the stock market index at the time of the purchase. Clearly some types of contextual information can be more relevant in a given application than some other types. For example, in this example, the value of a stock market is likely to be much less relevant as contextual information than the purpose of buying a book.

Because the relevance of contextual factors can vary dramatically from application to application (e.g., location as a recommendation context may matter significantly in one recommendation application, but have absolutely no impact in another), domain expertise typically plays a big role in identifying a candidate set of contextual factors for a given application. For example, for mobile recommendation applications, the following four general types of contextual information are often considered [5, 43]: *physical context* (e.g., time, position, activity of the user, weather, light conditions, temperature), *social context* (e.g., is the user alone or

in the group, presence and role of other people around the user), *interaction media context* (e.g., device characteristics—phone/tablet/laptop/etc., media content type—text/audio/video/etc.), *modal context* (e.g., user’s state of mind—cognitive capabilities, mood, experience, current goals).

In addition to using the *manual* approach, e.g., leveraging domain knowledge of the recommender system’s designer or a market expert in a given application domain, there are also several *computational* approaches to determining the relevance of a given type of contextual information. In particular, numerous existing feature selection procedures from machine learning [57], data mining [63], and statistics [36] can be used during the data preprocessing phase, based on existing ratings data. One methodology of deciding which contextual attributes should be used in a recommendation application (and which should not) is presented in [6]. In particular, Adomavicius et al. [6] propose that a wide range of contextual attributes should be initially selected by the domain experts as possible candidates for the contextual attributes for the application. For example, in a movie recommendation application described in Example 6.1, we can initially consider such contextual attributes as Time, Theater, Companion, Weather, as well as a broad set of other contextual attributes that can possibly affect the movie-watching experiences, as initially identified by the domain experts for the application. Then, after collecting the data, including the rating data and the contextual information, we may apply various types of statistical tests identifying which of the chosen contextual attributes are truly significant in the sense that they indeed affect movie-watching experiences, as manifested by significant deviations in ratings across different values of a contextual attribute. For example, we may apply pairwise t-tests to see if good weather vs. bad weather or seeing a movie alone vs. with a companion significantly affect the movie-watching experiences (as indicated by statistically significant changes in rating distributions). This procedure provides an example of screening all the initially considered contextual attributes and filtering out those that do not matter for a particular recommendation application. For example, we may conclude that the Time, Theater and Companion contexts matter, while the Weather context does not in the considered movie recommendation application. The use of statistical tests for determining the relevance of contextual information has been further explored in subsequent research [67].

Another approach for assessing the relevance of contextual information has been proposed by Baltrunas et al. [20], who developed a survey-based instrument that asks the users to judge what their preferences would be in a wide variety of hypothetical (i.e., imagined) contextual situations. This allows to collect richer contextual preference information in a short timeframe, evaluate the impact of each contextual factor on user preferences based on the collected data, and include into the resulting context-aware system only those factors that were shown to be important. Even though the collected data includes only hypothetical contextual preferences (i.e., preferences for items that users imagined consuming under certain contextual circumstances), the authors demonstrate that the resulting context-aware recommender system was perceived to be more effective by users as compared to the non-context-aware recommender.

The detailed discussion of the specific contextual feature selection procedures is beyond the scope of this chapter; in the remainder of this chapter we will assume that only the relevant contextual information is stored in the data.

In this section we reviewed major approaches to modeling contextual information and also put extra focus on the most popular—representational—approach. Since it has been studied most extensively in the past, we will focus on the major paradigms for incorporating contextual information into the representational approach in the next section.

6.3 Paradigms for Incorporating Representational Context in Recommender Systems

Once the relevant context information has been identified and obtained using the representational approach to CARS, the next step is to use context intelligently in order to produce better recommendations. Different approaches to using contextual information in recommender systems can be broadly categorized into two groups: (1) recommendation via *context-driven querying and search*, and (2) recommendation via *contextual preference elicitation and estimation*. The context-driven querying and search approach has been used by a wide variety of mobile and travel/tourist recommender systems [3, 35, 86]. Systems using this approach typically use contextual information (obtained either directly from the user, e.g., by specifying current mood or interest, or from the environment, e.g., obtaining local time, weather, or current location) to query or search a certain repository of resources (e.g., restaurants) and present the resources that best match a given query (e.g., nearby restaurants that are currently open) to the user. One of the early examples of this approach is the Cyberguide project [3], which developed several *tour guide* prototypes for different hand-held platforms. Abowd et al. [3] discuss different architectures and features necessary to provide realistic tour guide services to mobile users and, more specifically, the role that the contextual knowledge of the user's current and past locations can play in the recommendation and guiding process. Among the many other examples of context-aware tourist guide systems proposed in research literature we can mention GUIDE [38], INTRIGUE [16], COMPASS [86], and MyMap [41] systems.

The other general approach to using contextual information in the recommendation process, i.e., via contextual preference elicitation and estimation, represents a more recent trend in context-aware recommender systems literature [6, 7, 68, 71, 89]. In contrast to the previously discussed context-driven querying and search approach (where the recommender systems typically use the current context information and specified current user's interest as queries to search for the most appropriate content), techniques that follow this second approach attempt to model and learn contextual user preferences, e.g., by observing the interactions of this and other users with the systems or by obtaining preference feedback from the user on various

previously recommended items. To model users' context-sensitive preferences and generate recommendations, these techniques typically either adopt existing collaborative filtering, content-based, or hybrid recommendation methods to context-aware recommendation settings or apply various intelligent data analysis techniques from data mining or machine learning (such as Bayesian classifiers or support vector machines).

While both general approaches offer a number of research challenges, in the remainder of this chapter we will focus on the second, more recent trend of the contextual preference elicitation and estimation in recommender systems. We do want to mention that it is possible to design applications that combine the techniques from both general approaches (i.e., both context-driven querying and search as well as contextual preference elicitation and estimation) into a single system. For example, the UbiquiTO system [35], which implements a mobile tourist guide, provides intelligent adaptation not only based on the specific context information, but also uses various rule-based and fuzzy set techniques to adapt the application content based on the user preferences and interests. Similarly, the News@hand system [34] uses semantic technologies to provide personalized news recommendations that are retrieved using user's concept-based queries or calculated according to a specific user's (or a user group's) profile.

To start the discussion of the contextual preference elicitation and estimation techniques for the representational approach to CARS, note that, in its general form, a traditional two-dimensional (2D) ($User \times Item$) recommender system can be described as a *function*, which takes partial user preference data as its *input* and produces a list of recommendations for each user as an *output*. Accordingly, Fig. 6.4 presents a general overview of the traditional 2D recommendation process, which includes three components: data (input), 2D recommender system (function), and recommendation list (output). Note that, as indicated in Fig. 6.4, after the recommendation function is defined (or constructed) based on the available data, recommendation list for any given user u is typically generated by using the recommendation function on user u and all candidate items to obtain a predicted rating for each of the items and then by ranking all items according to their predicted rating value. Later in this section, we will discuss how the use of contextual information in each of those three components gives rise to three different paradigms for context-aware recommender systems.

As mentioned in Sect. 6.2.2, traditional recommender systems are built based on the knowledge of *partial user preferences*, i.e., user preferences for some (often limited) set of items, and the input data for traditional recommender systems



Fig. 6.4 General components of the traditional recommendation process

is typically based on the records of the form $\langle user, item, rating \rangle$. In contrast, context-aware recommender systems are built based on the knowledge of *partial contextual user preferences* and typically deal with data records of the form $\langle user, item, context, rating \rangle$, where each specific record includes not only how much a given user liked a specific item, but also the context in which the item was consumed by this user (e.g., $Context = \text{Saturday}$). Also, in addition to the descriptive information about users (e.g., demographics), items (e.g., item features), and ratings (e.g., multi-criteria rating information), context-aware recommender systems may also make use of additional context attributes, such as context hierarchies (e.g., $\text{Saturday} \rightarrow \text{Weekend}$) mentioned in Sect. 6.2.2. Based on the presence of this additional contextual data, several important questions arise: How contextual information should be reflected when modeling user preferences? Can we reuse the wealth of knowledge in traditional (non-contextual) recommender systems to generate context-aware recommendations? We will explore these questions in this chapter in more detail.

In the presence of available contextual information, following the diagrams in Fig. 6.5, we start with the data having the form $U \times I \times C \times R$, where C is additional contextual dimension and end up with a list of contextual recommendations $i_1, i_2, i_3 \dots$ for each user. However, unlike the process in Fig. 6.4, which does not take into account the contextual information, we can apply the information about the current (or expected) context c at various stages of the recommendation process. More specifically, the context-aware recommendation process that is based

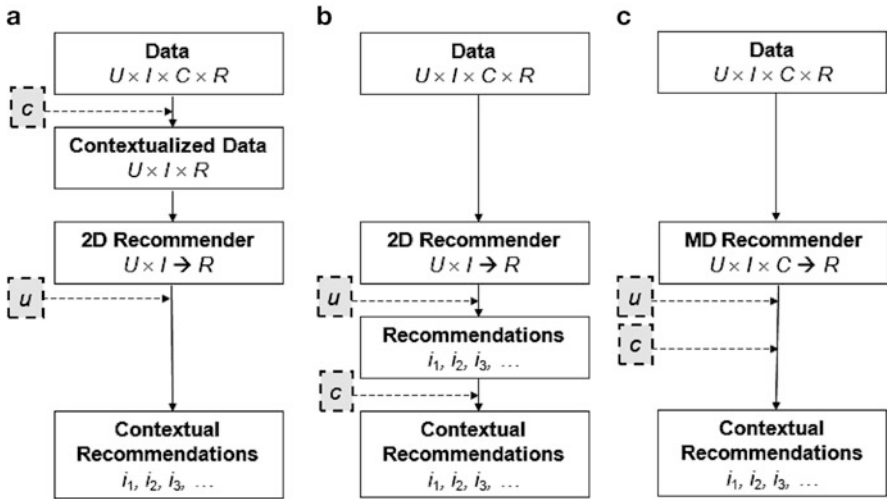


Fig. 6.5 Paradigms for incorporating context in recommender systems. (a) Contextual pre-filtering; (b) contextual post-filtering; (c) contextual modeling

on contextual user preference elicitation and estimation can take one of the three forms, based on which of the three components the context is used in, as shown in Fig. 6.5:

- *Contextual pre-filtering* (or contextualization of recommendation input). In this recommendation paradigm (presented in Fig. 6.5a), contextual information drives data selection or data construction for that specific context. In other words, information about the current context c is used for selecting or constructing the relevant set of data records (i.e., ratings). Then, ratings can be predicted using any traditional 2D recommender system on the selected data.
- *Contextual post-filtering* (or contextualization of recommendation output). In this recommendation paradigm (presented in Fig. 6.5b), contextual information is initially ignored, and the ratings are predicted using any traditional 2D recommender system on the *entire* data. Then, the resulting set of recommendations is adjusted (*contextualized*) for each user using the contextual information.
- *Contextual modeling* (or contextualization of recommendation function). In this recommendation paradigm (presented in Fig. 6.5c), contextual information is used directly in the modeling technique as part of rating estimation.

In the remainder of this section we will discuss these three approaches in detail.

6.3.1 Contextual Pre-filtering

As shown in Fig. 6.5a, the contextual pre-filtering approach uses contextual information to select or construct the most relevant 2D ($User \times Item$) data for generating recommendations. One major advantage of this approach is that it allows deployment of any of the numerous traditional recommendation techniques previously proposed in the literature [9]. In particular, in one possible use of this approach, context c essentially serves as a *query* for selecting (filtering) relevant ratings data. An example of a contextual data filter for a movie recommender system would be: if a person wants to see a movie on Saturday, *only* the Saturday rating data is used to recommend movies. Note that this example represents an *exact pre-filter*. In other words, the data filtering query has been constructed using exactly the specified context.

For example, following the contextual pre-filtering paradigm, Adomavicius et al. [6] proposed a *reduction-based approach*, which reduces the problem of multidimensional (MD) contextual recommendations to the standard 2D $User \times Item$ recommendation space. Therefore, as with any contextual pre-filtering approach, one important benefit of the reduction-based approach is that all the previous research on 2D recommender systems is directly applicable in the MD case after the reduction is done. In particular, let $R_{User \times Item}^D: U \times I \rightarrow Rating$ be any 2D rating estimation function that, given existing ratings D (i.e., D contains records $\langle user, item, rating \rangle$ for each of the known, user-specified ratings), can calculate a prediction for any rating, e.g., $R_{User \times Item}^D(John, StarWars)$. A three-dimensional rating prediction function supporting the context of time can be defined similarly as $R_{User \times Item \times Time}^D: U \times I \times T \rightarrow Rating$, where D contains records $\langle$

$user, item, time, rating >$ for the user-specified ratings. Then the three-dimensional prediction function can be expressed through a 2D prediction function in several ways, including:

$$\forall (u, i, t) \in U \times I \times T, R_{User \times Item \times Time}^D(u, i, t) = R_{User \times Item}^{D[Time=t](User, Item, Rating)}(u, i).$$

Here $[Time = t]$ denotes a simple contextual pre-filter, and $D[Time = t](User, Item, Rating)$ denotes a rating dataset obtained from D by selecting only the records where $Time$ dimension has value t and keeping only the values for $User$ and $Item$ dimensions, as well as the value of the rating itself. That is, if we treat a dataset of three-dimensional ratings D as a relation, then $D[Time = t](User, Item, Rating)$ is simply another relation obtained from D by performing two relational operations: selection and, subsequently, projection.

However, the exact context sometimes can be too narrow. Consider, for example, the context of watching a movie with a girlfriend in a movie theater on Saturday or, i.e., $c = (Girlfriend, Theater, Saturday)$. Using this exact context as a data filtering query may be problematic for several reasons. First, certain aspects of the overly specific context may not be significant. For example, user's movie watching preferences with a girlfriend in a theater on Saturday may be exactly the same as on Sunday, but different from Wednesday's. Therefore, it may be more appropriate to use a more general context specification, i.e., *Weekend* instead of *Saturday*. And second, exact context may not have enough data for accurate rating prediction, which is known as the "sparsity" problem in recommender systems literature. In other words, the recommender system may not have enough data points about the past movie watching preferences of a given user with a girlfriend in a theater on Saturday.

Adomavicius et al. [6] introduce the notion of *generalized pre-filtering*, which allows to generalize the data filtering query obtained based on a specified context. More formally, let's define $c' = (c'_1, \dots, c'_k)$ to be a generalization of context $c = (c_1, \dots, c_k)$ if and only if $c_i \rightarrow c'_i$ for every $i = 1, \dots, k$ in the corresponding context hierarchy. Then, c' (instead of c) can be used as a data query to obtain contextualized ratings data.

Following the idea of context generalization, Adomavicius et al. [6] proposed to use not a simple pre-filter $[Time = t]$, which represents the exact context t of the rating (u, i, t) , but rather a generalized pre-filter $[Time \in S_t]$, where S_t denotes some superset of context t . Here S_t is called a *contextual segment* [6]. For example, if we would like to predict how much John Doe would like to see the "Gladiator" movie on Monday, i.e., to calculate $R_{User \times Item \times Time}^D(JohnDoe, Gladiator, Monday)$, we could use not only other user-specified *Monday* ratings for prediction, but *Weekday* ratings in general. In other words, for every (u, i, t) where $t \in Weekday$, we can predict the rating as $R_{User \times Item \times Time}^D(u, i, t) = R_{User \times Item}^{D[Time \in Weekday](User, Item, AGGR(Rating))}(u, i)$. More generally, in order to estimate some rating $R(u, i, t)$, we can use some specific contextual segment S_t as: $R_{User \times Item \times Time}^D(u, i, t) = R_{User \times Item}^{D[Time \in S_t](User, Item, AGGR(Rating))}(u, i)$.

Note, that we have used the $AGGR(Rating)$ notation in the above expressions, since there may be several user-specified ratings with the same *User* and *Item* values for different *Time* instances in dataset D belonging to some contextual segment S_i (e.g., different ratings for *Monday* and *Tuesday*, all belonging to segment *Weekday*). Therefore, we have to aggregate these values using some aggregation function, e.g., averaging, when reducing the dimensionality of the recommendation space. The above three-dimensional reduction-based approach can be extended to a general pre-filtering method reducing an arbitrary n -dimensional recommendation space to an m -dimensional one (where $m < n$). In this chapter we will assume that $m = 2$ because traditional recommendation algorithms are only designed for the two-dimensional *User* $\times$ *Item* case. Note that there typically exist multiple different possibilities for context generalization, based on the context taxonomy and the desired context granularity. For example, let's assume that we have the following contextual taxonomies (*is-a* or *belongs-to* relationships) that can be derived from context hierarchies:

- *Company*: Girlfriend $\rightarrow$ Friends $\rightarrow$ NotAlone $\rightarrow$ AnyCompany;
- *Place*: Theater $\rightarrow$ AnyPlace;
- *Time*: Saturday $\rightarrow$ Weekend $\rightarrow$ AnyTime.

Then, the following are just several examples of possible generalizations c' of the above-mentioned context $c = (\text{Girlfriend}, \text{Theater}, \text{Saturday})$:

- $c' = (\text{Girlfriend}, \text{AnyPlace}, \text{Saturday})$;
- $c' = (\text{Friends}, \text{Theater}, \text{AnyTime})$;
- $c' = (\text{NotAlone}, \text{Theater}, \text{Weekend})$.

Therefore, choosing the “right” generalized pre-filter becomes an important problem. One option is to use a manual, expert-driven approach; e.g., always generalize specific days of week into more general *Weekday* or *Weekend*. Another option is to use a more automated approach, which could empirically evaluate the predictive performance of the recommender system on contextualized input datasets obtained from each generalized pre-filter, and then would automatically choose the pre-filter with best performance. An interesting and important research issue is how to deal with potential computational complexity of this approach due to context granularity; in other words, in cases of applications with highly granular contexts, there may exist a very large number of possible context generalizations, for which exhaustive search techniques would not be practical. For such cases, effective greedy approaches would need to be developed. Jiang and Tuzhilin [50] examine optimal levels of granularity of customer segments in order to maximize predictive performance of segmentation methods. Applicability of these techniques in the context-aware recommender systems settings constitutes an interesting problem for future research.

So far, we have discussed applying only one pre-filter at a time. However, as it has been well-documented in recommender systems literature, often a combination (a “blend” or an ensemble) of several solutions provides significant performance improvements over the individual approaches [30, 31, 58, 74]. Therefore, it may also

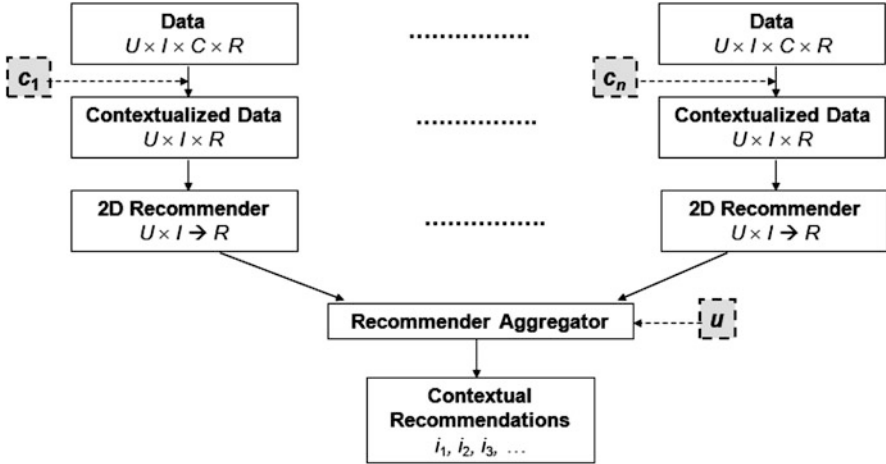


Fig. 6.6 Combining multiple pre-filters

be useful to combine several contextual pre-filters into one model at the same time. The rationale for having a number of different pre-filters is based on the fact that, as mentioned earlier, typically there can be multiple different (and potentially relevant) generalizations of the same specific context. Following this idea, Adomavicius et al. [6] use pre-filters based on the number of possible contexts for each rating, and then combine recommendations resulting from each contextual pre-filter. The general overview of this approach is shown in Fig. 6.6. Note that the combination of several pre-filters can be done in multiple ways. For example, for rating estimation in a specific context, (a) one could choose the best-performing pre-filter, or (b) use an aggregate prediction from the entire “ensemble” of pre-filters.

Also note that the contextual pre-filtering approach is related to the problems of building local models in machine learning and data mining [13]. Rather than building the global rating estimation model utilizing all the available ratings, this approach builds a *local* rating estimation model that uses only the ratings pertaining to the user-specified criteria in which a recommendation is made (e.g., morning). It is important to know if a local model generated by the pre-filtering approach outperforms the global model of the traditional 2D technique, where all the information associated with the contextual dimensions is simply ignored. For example, it is possible that it is better to use the contextual pre-filtering to recommend movies to see in the movie theaters on weekends, but use the traditional 2D technique for movies to see at home. This is the case because pre-filtering, on the one hand, focuses recommendations on a *particular segment* and builds a local prediction model for this segment, but, on the other hand, computes these recommendations based on a *smaller* number of points limited to the considered segment. This tradeoff between having *more relevant* data for calculating an unknown rating based only on the ratings with the same or similar context and

having *fewer* data points used in this calculation belonging to a particular segment (i.e., the *sparsity* effect) explains why the pre-filtering recommendation method can outperform traditional 2D recommendation techniques on some segments and underperform on others. Which of these two trends dominates on a particular segment may depend on the application domain and on the specifics of the available data. This observation provides yet another motivation behind the aforementioned ensemble-based approaches [6] that can combine a number of contextual pre-filters along with the traditional 2D technique (i.e., as a default filter, where no filtering is done). This allows to take advantage of more-targeted (local) and well-performing contextual segments where possible, while reverting to the default, traditional 2D technique in other cases.

Among other developments, Ahn et al. [12] use a technique similar to the contextual pre-filtering to recommend advertisements to mobile users by taking into account user location, interest, and time, and Lombardi et al. [64] evaluate the effect of contextual information using a pre-filtering approach on the data obtained from an online retailer. Also, Baltrunas and Ricci [22, 23] take a somewhat different approach to contextual pre-filtering in proposing and evaluating the benefits of the *item splitting* technique, where each item is split into several fictitious items based on the different contexts in which these items can be consumed. Similarly to the item splitting idea, Baltrunas and Amatriain [17] introduce the idea of *micro-profiling* (or user splitting), which splits the user profile into several (possibly overlapping) sub-profiles, each representing the given user in a particular context. The predictions are done using these contextual micro-profiles instead of a single user model. Note that these data construction techniques fit well under the contextual pre-filtering paradigm, because they are following the same basic idea (as the data filtering techniques described earlier)—using contextual information to reduce the problem of multidimensional recommendations to the standard 2D *User* $\times$ *Item* space, which then allows to use any traditional recommendation techniques for rating prediction. The idea of generalized contextual pre-filtering has also been adopted in various studies; for example, Zheng et al. [90] use a similar approach (called context “relaxation”) for travel recommendations. Furthermore, Codina et al. [40] provide a different approach to generalize the pre-filtering approach; they leverage semantic similarities between different contextual conditions and compute the recommendations based on the ratings taken not just from the single contextual condition, but from the similar contexts as well.

The idea of combining multiple filters is not limited only to pre-filtering and can be extended to post-filtering and contextual modeling approaches (which will be discussed next). In particular, complex contextual information can be split into several components, and the utility of each piece of contextual information may be different depending on whether it is used in the pre-filtering, post-filtering, or modeling stage. For example, time information (weekday vs. weekend) may be most useful to pre-filter relevant data, but weather information (sunny vs. rainy) may be the most appropriate to use as a post-filter. Determining the utility of different contextual information with respect to different paradigms of context-

aware recommender systems constitutes an interesting and promising direction for future research.

6.3.2 Contextual Post-filtering

As shown in Fig. 6.5b, the contextual post-filtering approach ignores context information in the input data when generating recommendations, i.e., when generating the ranked list of all candidate items from which any number of top- N recommendations can be made, depending on specific values of N . Then, the contextual post-filtering approach adjusts the obtained recommendation list for each user using context information. The recommendation list adjustments can be made by:

- *Filtering* out recommendations that are irrelevant (in a given context), or
- *Adjusting* the ranking of recommendations on the list (based on a given context).

For example, in a movie recommendation application, if a person wants to see a movie on a weekend, and on weekends she only watches comedies, the system can filter out all non-comedies from the recommended movie list. More generally, the basic idea for contextual post-filtering approaches is to analyze the contextual preference data for a given user in a given context to find specific item usage patterns (e.g., user Jane Doe watches only comedies on weekends) and then use these patterns to adjust the item list, resulting in more “contextual” recommendations, as depicted in Fig. 6.7.

As with many recommendation techniques, the contextual post-filtering approaches can be classified into heuristic and model-based techniques. *Heuristic* post-filtering approaches focus on finding common item characteristics (attributes) for a given user in a given context (e.g., preferred actors to watch in a given context), and then use these attributes to adjust the recommendations, including:

- *Filtering* out recommended items that do not have a significant number of these characteristics (e.g., to be recommended, the movies must have at least two of the preferred actors in a given context), or

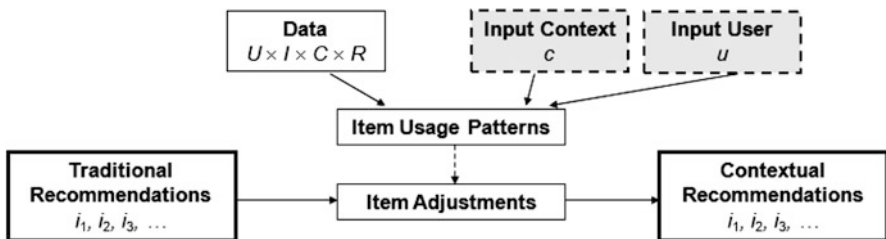


Fig. 6.7 Final phase of the contextual post-filtering approach: recommendation list adjustment

- *Ranking* recommended items based on how many of these relevant characteristics they have (e.g., the movies that star more of the user's preferred actors in a given context will be ranked higher).

In contrast, *model-based* post-filtering approaches can build predictive models that calculate the probability with which the user chooses a certain type of item in a given context, i.e., probability of relevance (e.g., likelihood of choosing movies of a certain genre in a given context), and then use this probability to adjust the recommendations, including:

- *Filtering* out recommended items that have the probability of relevance smaller than a pre-defined minimal threshold (e.g., remove movies of genres that have a low likelihood of being picked), or
- *Ranking* recommended items by weighting the predicted rating with the probability of relevance.

Panniello et al. [71] provide an experimental comparison of the exact pre-filtering method (discussed in Sect. 6.3.1) versus two different post-filtering methods—*Weight* and *Filter*—using several real-world e-commerce datasets. The *Weight* post-filtering method reorders the recommended items by weighting the predicted rating with the probability of relevance in that specific context, and the *Filter* post-filtering method filters out recommended items that have small probability of relevance in the specific context. Interestingly, the empirical results show that the *Weight* post-filtering method dominates the exact pre-filtering, which in turn dominates the *Filter* post-filtering method, thus, indicating that the best approach to use (pre- or post-filtering) really depends on a given application.

As was the case with the contextual pre-filtering approach, a major advantage of the contextual post-filtering approach is that it allows using any of the numerous traditional recommendation techniques previously proposed in the literature [9]. Also, similarly to the contextual pre-filtering approaches, incorporating context generalization techniques into post-filtering techniques constitutes an interesting issue for future research.

6.3.3 Contextual Modeling

As shown in Fig. 6.5c, the contextual modeling approach uses contextual information *directly* in the recommendation function as an explicit predictor of a user's rating for an item. While contextual pre-filtering and post-filtering approaches can use traditional 2D recommendation functions, the contextual modeling approach gives rise to truly multidimensional recommendation functions, which essentially represent predictive models (built using decision trees, regressions, probabilistic models, or other techniques) or heuristic calculations that incorporate contextual information in addition to the user and item data, i.e., $Rating = R(User, Item, Context)$. A significant number of recommendation algorithms—based on a variety of heuristics as

well as predictive modeling techniques—have been developed over the last 15 years, and some of these techniques can be extended from the 2D to the multidimensional recommendation settings.

For example, Adomavicius and Tuzhilin [8] proposes to extend the traditional two-dimensional (2D) neighborhood-based approach [28, 81] to the multidimensional case, which includes the contextual information, by using an n -dimensional distance metric instead of the user-user or item-item similarity metrics traditionally used in such techniques. To illustrate how this can be done, consider an example of the three-dimensional $User \times Item \times Time$ recommendation space. Following the traditional nearest neighbor heuristic that is based on the weighted sum of relevant ratings, the prediction of a specific rating $r_{u,i,t}$ in this example can be expressed as:

$$r_{u,i,t} = k \sum_{(u',i',t') \neq (u,i,t)} W((u,i,t), (u',i',t')) \times r_{u',i',t'},$$

where $W((u,i,t), (u',i',t'))$ describes the “weight” that rating $r_{u',i',t'}$ carries in the prediction of $r_{u,i,t}$, and k is a normalizing factor. Weight $W((u,i,t), (u',i',t'))$ is typically inversely related to the distance between points (u,i,t) and (u',i',t') in multidimensional space, i.e., $dist[(u,i,t), (u',i',t')]$. In other words, the closer the two points are (i.e., the smaller the distance between them), the more weight $r_{u',i',t'}$ carries in the weighted sum. Moreover, Adomavicius and Tuzhilin [8] presents different types of the distance functions used for defining weights $W((u,i,t), (u',i',t'))$; see [8] for additional details.

Another heuristic-based contextual modeling (CM) method is presented in [70], where four variants of the same CM method are considered, i.e., Mdl1, Mdl2, Mdl3, Mdl4. Each of these CM methods requires building a contextual profile $prof(u, c)$ for user u in context c , and then using the contextual profiles of all the users to find the N nearest neighbors of user u in terms of these profiles in context c . The four types of the CM approaches (Mdl1, Mdl2, Mdl3, Mdl4) vary in the constraints by which the neighbors are selected. In Mdl1, there is no constraint in the selection of the N neighbors, i.e., they can be selected based on any context at any level of the hierarchy. In Mdl2, an equal proportion of neighbors is selected from each context c regardless of the context hierarchy. In Mdl3, N neighbors are selected from each context c and each level of the context hierarchy. In Mdl4, an equal proportion of neighbors is selected from each context c at the same level of context hierarchy.

In addition to the heuristic-based contextual modeling techniques, there have been several model-based techniques proposed in the literature. For example, Adomavicius and Tuzhilin [8] present a method of extending a regression-based Hierarchical Bayesian (HB) collaborative filtering model of estimating unknown ratings proposed by Ansari et al. [15] in order to incorporate additional contextual dimensions, such as time and location, into the HB model.

Similarly, Karatzoglou et al. [52] have proposed an extended version of matrix factorization approach based on tensor factorization, which allows to incorporate contextual information into the recommendation process. A alternative matrix-factorization-based approach has been proposed by Baltrunas et al. [21], who

extended the traditional matrix factorization technique to context-aware settings by introducing additional model parameters to model the interaction of the contextual factors with ratings.

In addition to extending the traditional *User* $\times$ *Item* model-based collaborative filtering techniques to incorporate the contextual dimensions, there have also been some new techniques developed specifically for context-aware recommender systems based on the context modeling paradigm. For example, following the general contextual modeling paradigm, Oku et al. [68] propose to incorporate additional contextual dimensions (such as time, companion, and weather) directly into recommendation space and use machine learning technique to provide recommendations in a restaurant recommender system. In particular, they use support vector machine (SVM) classification method, which views the set of liked items and the set of disliked items of a user in various contexts as two sets of vectors in an n -dimensional space, and constructs a separating hyperplane in this space, which maximizes the separation between the two data sets. The resulting hyperplane represents a classifier for future recommendation decisions (i.e., a given item in a specific context will be recommended if it falls on the “like” side of the hyperplane, and will not be recommended if it falls on the “dislike” side). Furthermore, Oku et al. [68] empirically show that context-aware SVM significantly outperforms non-contextual SVM-based recommendation algorithm in terms of predictive accuracy and user’s satisfaction with recommendations. Similarly, Yu et al. [89] use contextual modeling approach to provide content recommendations for smartphone users by introducing context as additional model dimensions and using hybrid recommendation technique (synthesizing content-based, Bayesian-classifier, and rule-based methods) to generate recommendations. Also, Hariri et al. [46] employ the Latent Dirichlet Allocation (LDA) model for use in context-aware recommender systems, which allows to model jointly the users, items, and the meta-data associated with contextual information.

Finally, another model-based approach is presented in [2] where a Personalized Access Model (PAM) is presented that provides a set of personalized context-based services, including context discovery, contextualization, binding and matching services. Then Abbar et al. [2] describe how these services can be combined to form Context-Aware Recommender Systems (CARS) and deployed in order to provide superior context-aware recommendations.

In this section we described various ways to incorporate contextual information into recommendation algorithms within the framework of pre-filtering, post-filtering, and contextual modeling paradigms. Since CARS is still a relatively young sub-area of recommender systems, developing further improvements across all these three paradigms represent a promising research direction.

6.4 Discussion and Conclusions

Context-awareness is being recognized as an important issue in many recommendation applications, which is evidenced by an increasing number of papers on context-aware recommender systems that appear in conferences and journals, as

well as by the number and variety of research workshops that have been dedicated specifically to the topics related to context-aware recommender systems. Looking at the current state of the art in context-aware recommender systems, the main research issues, challenges, and directions can be broadly classified into the following four general categories [4]:

- *Fundamentals*, i.e., understanding the notion of context and modeling context in recommender systems.
- *Algorithms*, i.e., developing recommendation algorithms that can incorporate contextual information into recommender systems in advantageous ways.
- *Evaluation*, i.e., in-depth performance evaluation of various context-aware recommendation approaches and techniques, their benefits and limitations.
- *Engineering*, i.e., designing general-purpose architectures, frameworks, and approaches to facilitate the development, implementation, deployment, and use of context-aware recommendation capabilities.

Not surprisingly, the overwhelming majority of existing research on context-aware recommender systems can be classified under the “Algorithms” category [4], i.e., researchers have focused primarily on how to take advantage of contextual information in order to improve the quality of recommendations for different recommendation tasks and applications. Compared to “Algorithms” the other three categories have been relatively under-explored, although there have been more work in the several other areas in recent years.

One recent representative example in the “Evaluation” category is the work by Panniello et al. [70], who performed a comprehensive evaluation and comparison of several contextual pre-filtering, post-filtering, and modeling techniques under variety of conditions, e.g., for different recommendation tasks (find-all vs. top-k), different recommendation utility metrics (accuracy vs. diversity), the granularity of the processed contextual information, as well as other characteristics. Among many findings, the comparison shows that there is no “universally” best technique under all evaluation dimensions. For example, the contextual modeling and post-filtering approaches demonstrate good accuracy performance in many situations, while the exact pre-filtering approach often exhibits better diversity. However, the contextual modeling approaches tend to achieve a comparatively good balance in the accuracy-diversity trade-off. Another example in the “Evaluation” category is the work by Campos et al. [33], who focused on exploring “time” as one of the most valuable and widely used contextual factors in many recommender systems applications. For example, the authors review common evaluation practices and methodological issues related to the comparative evaluation of time-aware recommender systems. They also demonstrate that the choice of the evaluation conditions impacts the performance ranking of different recommendation strategies and propose a methodological framework for a robust and fair evaluation process. These works represent an important step in the direction of improved, standardized and, thus, more reproducible evaluation procedures for context-aware recommender systems.

Developing several large-scale publicly available contextual recommendation datasets for evaluation purposes would provide further stimulus for the CARS research activities and, therefore, this should also be an important priority for the research community.

However, additional research is needed not only in terms of purely algorithmic evaluation, but also in terms of user studies for understanding various behavioral and economic implications of using contextual information in recommender systems (e.g., user acceptance, satisfaction, intrusiveness, privacy, trust, etc.) and preferably not only on the offline data but also in live experiments on real users (i.e., so called A/B testing). One example of such effort is the work by Gorgoglione et al. [44], where the authors study the effects of contextual recommendations on the purchasing behavior of customers and their trust in the provided recommendations, as opposed to the usual predictive accuracy metrics used in most of the other CARS studies. In particular, Gorgoglione et al. [44] describe live controlled experiments performed by the authors with real customers of a major commercial European retailer, in which the authors compare recommendation accuracy, diversity, customers' purchasing behavior and measure customer trust in the provided recommendations across the contextual, content-based, and random recommendations. The authors show that context-aware recommendation techniques outperform traditional (non-contextual) approaches in terms of accuracy, trust, and several economics-based performance metrics across most of their experimental settings. Another interesting example of user studies with context-aware recommender systems is the work by Braunhofer et al. [26]. In this study, the users, after receiving a recommendation from a context-aware system that recommends points-of-interest, were asked to evaluate the system's performance on the following two dimensions: "Does this recommendation fit my preference?" (i.e., "personalization" performance) and "Is this recommendation well-chosen for the situation?" (i.e., "contextualization" performance). In this specific study, the authors show that their proposed technique is able to improve the baseline performance along one of these dimensions—contextualization. Exploring the possible relationships between these two performance dimensions further as well as understanding the performance of other context-aware recommendation techniques with respect to these dimensions represent interesting research directions. In summary, it is important for the CARS community to continue the lines of work described in [26, 44] and to provide strong additional evidence (e.g., via live controlled experiments) of the economic and usability advantages of CARS over the traditional recommendation methods.

Much of the work on context-aware recommender systems has been conceptual, where recommendation techniques are developed, tested on some (often limited) data, and shown to perform well in comparison to certain benchmarks. Historically, there has been little work done on the "Engineering" aspect for CARS, i.e., developing novel data structures, efficient storage methods, and new system architectures. A recent representative example of such system-building work is the study by Hussein et al. [47, 48], where the authors introduce a service-oriented architecture enabling to define and implement a variety of different "building blocks" for context-aware recommender systems, such as recommendation algorithms, context

sensors, various filters and converters, in a modular fashion. These building blocks can then be combined and reused in many different ways into systems that can generate contextual recommendations. Another example of such work is Abbar et al. [2], where the authors present a service-oriented approach that implements the Personalized Access Model (PAM) previously proposed by the authors. The implementation is done using global software architecture of CARS developed by the authors and described in [2]. In addition, there has been a number of recent research advances in database community in developing frameworks and techniques for building recommender systems functionality (including some context-aware functionality) directly into relational database engines [61, 62, 82, 83], which provides a number of important benefits in terms of storage, query processing, query optimization, and others.

Another important “Engineering” aspect is to develop richer interaction capabilities with CARS that make recommendations more flexible. As compared to traditional recommender systems, context-aware recommenders have two important differences. The first is *increased complexity*, since CARS involve not only users and items in the recommendation process, but also various types of contextual information. Thus, the types of recommendations can be significantly more complex in comparison to the traditional non-contextual cases. For example, in a movie recommendation application, a certain user (e.g., Tom) may seek recommendations for him and his girlfriend of top 3 movies and the best times to see them over the weekend. The second difference is *increased interactivity*, since more information (i.e., context) usually needs to be elicited from the user in the CARS settings. For example, to utilize the available contextual information, a CARS system may need to elicit from the user (Tom) with whom he wants to see a movie (e.g., girlfriend) and when (e.g., over the weekend) before providing any context-specific recommendations. The combination of these two features calls for the development of more flexible recommendation methods that allow the user to express the types of recommendations that are of interest to them rather than being “hard-wired” into the recommendation engines provided by most of the current vendors that, primarily, focus on recommending top- N items to the user and vice versa. The second requirement of interactivity also calls for the development of tools allowing users to provide inputs into the recommendation process in an interactive and iterative manner, preferably via some well-defined user interface (UI).

Such flexible context-aware recommendations can be supported in several ways. First, Adomavicius et al. [11] developed a *recommendation query language* REQUEST that allows its users to express in a flexible manner a broad range of recommendations that are tailored to their own individual needs and, therefore, more accurately reflect their interests. REQUEST is based on the multidimensional contextual recommendation model described in Sect. 6.2.2 and also in [6]. REQUEST supports a wide variety of features, and the interested reader can find the detailed account of these features as well as the formal syntax and various properties of the language in [11]. In addition, Adomavicius et al. [11] provide a discussion of the expressive power of REQUEST and present a multidimensional recommendation algebra that provides the theoretical basis for this language.

Another proposal to provide flexible recommendations is presented in [59], where the FlexRecs system and framework are described. FlexRecs approach supports flexible recommendations over structured data by decoupling the definition of a recommendation process from its execution. In particular, a recommendation can be expressed declaratively as a high-level parameterized workflow containing traditional relational operators and novel recommendation-specific operators that are combined together into a recommendation workflow.

Furthermore, since a major argument for introducing any recommendation query language is its use by the end-users, it is also very important to develop simple, friendly, interactive, and expressive user interfaces (UIs) for supporting flexible but sometimes complex contextual recommendations. High-quality UIs should reduce the complexity and simplify interactions between the end-users and the recommender system and make them available to wider audiences. For instance, for FlexRecs this may entail building a UI for defining and managing recommendation workflows, while for REQUEST this may involve building front-end UI allowing users to express REQUEST queries using visual and interactive methods. Developing these and other UIs for CARS constitutes a topic of future research.

“Fundamentals” represents arguably the least developed research direction. Context is a complex notion, and there are many diverse approaches of conceptualizing context in recommender systems, some of them still being debated among the researchers. Although the recommender systems community is gradually converging toward the common definition of context, and this chapter represents an attempt to integrate different approaches into one common framework, additional work is still required for the CARS community to arrive at a clear, more formalized definition of context and, therefore, many important questions still remain to be explored in a principled manner. For example, what are underlying theoretical underpinnings for context relevance? Are there systematic approaches for identifying relevant contextual factors (i.e., on which to collect data)? When explicit contextual information is not available, when should we model context as a latent factor vs. ignore modeling the context altogether? What are the tradeoffs of different modeling assumptions (e.g., static vs. dynamic context)? Most of existing research on context-aware recommender systems follows the assumption that contextual dimensions are stable and observable, but implications of other modeling assumptions and approaches should also be explored. These are only some of the examples of various questions that the CARS community can explore further.

In summary, the field of context-aware recommender systems (CARS) is still a relatively new and promising area of research with many interesting and practically important research problems, and much more work is needed to investigate them in a principled and comprehensive manner.

Acknowledgement Research of A. Tuzhilin was supported in part by the National Science Foundation grant IIS-1256036, USA.

References

1. AWS re:Invent 2012, Day 1 Keynote. <http://www.youtube.com/watch?v=8FJ5DBLSFe4>. YouTube video; Accessed: 2014-06-28
2. Abbar, S., Bouzeghoub, M., Lopez, S.: Context-aware recommender systems: A service-oriented approach. *VLDB PersDB Workshop* (2009)
3. Abowd, G.D., Atkeson, C.G., Hong, J., Long, S., Kooper, R., Pinkerton, M.: Cyberguide: A mobile context-aware tour guide. *Wireless Networks* **3**(5), 421–433 (1997)
4. Adomavicius, G., Jannach, D.: Preface to the special issue on context-aware recommender systems. *User Model. User-Adapt. Interact.* **24**(1–2), 1–5 (2014)
5. Adomavicius, G., Mobasher, B., Ricci, F., Tuzhilin, A.: Context-aware recommender systems. *AI Magazine* **32**(3), 67–80 (2011)
6. Adomavicius, G., Sankaranarayanan, R., Sen, S., Tuzhilin, A.: Incorporating contextual information in recommender systems using a multidimensional approach. *ACM Transactions on Information Systems (TOIS)* **23**(1), 103–145 (2005)
7. Adomavicius, G., Tuzhilin, A.: Multidimensional recommender systems: A data warehousing approach. In: L. Fiege, G. Mhl, U. Wilhelm (eds.) *Electronic Commerce, Lecture Notes in Computer Science*, vol. 2232, pp. 180–192. Springer Berlin Heidelberg (2001). DOI [10.1007/3-540-45598-1_17](https://doi.org/10.1007/3-540-45598-1_17). URL http://dx.doi.org/10.1007/3-540-45598-1_17
8. Adomavicius, G., Tuzhilin, A.: Incorporating context into recommender systems using multidimensional rating estimation methods. In: *Proceedings of the 1st International Workshop on Web Personalization, Recommender Systems and Intelligent User Interfaces (WPRSIUI 2005)* (2005)
9. Adomavicius, G., Tuzhilin, A.: Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* **17**(6), 734–749 (2005)
10. Adomavicius, G., Tuzhilin, A.: Context-aware recommender systems. In: Ricci et al. [79], pp. 217–253
11. Adomavicius, G., Tuzhilin, A., Zheng, R.: REQUEST: A query language for customizing recommendations. *Information System Research* **23**(1), 99–117 (2011)
12. Ahn, H., Kim, K., Han, I.: Mobile advertisement recommender system using collaborative filtering: MAR-CF. In: *Proceedings of the 2006 Conference of the Korea Society of Management Information Systems*,
13. Alpaydin, E.: *Introduction to machine learning*. The MIT Press (2004)
14. Anand, S.S., Mobasher, B.: Contextual recommendation. *WebMine, LNAI* **4737**, 142–160 (2007)
15. Ansari, A., Essegai, S., Kohli, R.: Internet recommendation systems. *Journal of Marketing Research* **37**(3), 363–375 (2000)
16. Ardissono, L., Goy, A., Petrone, G., Segnan, M., Torasso, P.: Intrigue: personalized recommendation of tourist attractions for desktop and hand held devices. *Applied Artificial Intelligence* **17**(8), 687–714 (2003)
17. Baltrunas, L., Amatriain, X.: Towards time-dependant recommendation based on implicit feedback. In: *Workshop on Context-Aware Recommender Systems (CARS 2009)*. New York (2009)
18. Baltrunas, L., Kaminskas, M., Ludwig, B., Moling, O., Ricci, F., Aydin, A., Lke, K.H., Schwaiger, R.: Incarmusic: Context-aware music recommendations in a car. In: C. Huemer, T. Setzer (eds.) *E-Commerce and Web Technologies, Lecture Notes in Business Information Processing*, vol. 85, pp. 89–100. Springer Berlin Heidelberg (2011). DOI [10.1007/978-3-642-23014-1_8](https://doi.org/10.1007/978-3-642-23014-1_8). URL http://dx.doi.org/10.1007/978-3-642-23014-1_8
19. Baltrunas, L., Ludwig, B., Peer, S., Ricci, F.: Context-aware places of interest recommendations for mobile users. In: A. Marcus (ed.) *Design, User Experience, and Usability. Theory, Methods, Tools and Practice, Lecture Notes in Computer Science*, vol. 6769, pp. 531–540. Springer Berlin Heidelberg (2011). DOI [10.1007/978-3-642-21675-6_61](https://doi.org/10.1007/978-3-642-21675-6_61). URL http://dx.doi.org/10.1007/978-3-642-21675-6_61

20. Baltrunas, L., Ludwig, B., Peer, S., Ricci, F.: Context relevance assessment and exploitation in mobile recommender systems. *Personal and Ubiquitous Computing* **16**(5), 507–526 (2012)
21. Baltrunas, L., Ludwig, B., Ricci, F.: Matrix factorization techniques for context aware recommendation. In: *Proceedings of the Fifth ACM Conference on Recommender Systems, RecSys '11*, pp. 301–304. ACM, New York, NY, USA (2011). DOI [10.1145/2043932.2043988](https://doi.org/10.1145/2043932.2043988). URL <http://doi.acm.org/10.1145/2043932.2043988>
22. Baltrunas, L., Ricci, F.: Context-dependent items generation in collaborative filtering. In: *Workshop on Context-Aware Recommender Systems (CARS 2009)*. New York (2009)
23. Baltrunas, L., Ricci, F.: Experimental evaluation of context-dependent collaborative filtering using item splitting. *User Modeling and User-Adapted Interaction* **24**(1–2), 7–34 (2014). DOI [10.1007/s11257-012-9137-9](https://doi.org/10.1007/s11257-012-9137-9). URL <http://dx.doi.org/10.1007/s11257-012-9137-9>
24. Bauman, K., Tuzhilin, A.: Discovering contextual information from user reviews for recommendation purposes. In: *Proceedings of the ACM RecSys Workshop on New Trends in Content Based Recommender Systems* (2014)
25. Bazire, M., Brézillon, P.: Understanding context before using it. In: A. Dey, et al. (eds.) *Proceedings of the 5th International Conference on Modeling and Using Context*. Springer-Verlag (2005)
26. Braunhofer, M., Elahi, M., Ge, M., Ricci, F.: Context dependent preference acquisition with personality-based active learning in mobile recommender systems. In: P. Zaphiris, A. Ioannou (eds.) *Learning and Collaboration Technologies. Technology-Rich Environments for Learning and Collaboration - First International Conference, LCT 2014, Held as Part of HCI International 2014, Heraklion, Crete, Greece, June 22–27, 2014, Proceedings, Part II, Lecture Notes in Computer Science*, vol. 8524, pp. 105–116. Springer (2014). DOI [10.1007/978-3-319-07485-6_11](https://doi.org/10.1007/978-3-319-07485-6_11). URL http://dx.doi.org/10.1007/978-3-319-07485-6_11
27. Braunhofer, M., Kaminskas, M., Ricci, F.: Recommending music for places of interest in a mobile travel guide. In: *Proceedings of the Fifth ACM Conference on Recommender Systems, RecSys '11*, pp. 253–256. ACM, New York, NY, USA (2011). DOI [10.1145/2043932.2043977](https://doi.org/10.1145/2043932.2043977). URL <http://doi.acm.org/10.1145/2043932.2043977>
28. Breese, J.S., Heckerman, D., Kadie, C.: Empirical analysis of predictive algorithms for collaborative filtering. In: *Proceedings of the Fourteenth Conference on Uncertainty in Artificial Intelligence*, vol. 461, pp. 43–52. San Francisco, CA (1998)
29. Bulander, R., Decker, M., Schiefer, G., Kolmel, B.: Comparison of different approaches for mobile advertising. In: *Proceedings of the Second IEEE International Workshop on Mobile Commerce and Services, WMCS '05*, pp. 174–182. IEEE Computer Society, Washington, DC, USA (2005). DOI [10.1109/WMCS.2005.8](https://doi.org/10.1109/WMCS.2005.8). URL <http://dx.doi.org/10.1109/WMCS.2005.8>
30. Burke, R.: Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction* **12**(4), 331–370 (2002)
31. Burke, R.: Hybrid web recommender systems. *The Adaptive Web* pp. 377–408 (2007)
32. Cai, R., Zhang, C., Wang, C., Zhang, L., Ma, W.Y.: Musicsense: Contextual music recommendation using emotional allocation modeling. In: *Proceedings of the 15th International Conference on Multimedia, MULTIMEDIA '07*, pp. 553–556. ACM, New York, NY, USA (2007). DOI [10.1145/1291233.1291369](https://doi.org/10.1145/1291233.1291369). URL <http://doi.acm.org/10.1145/1291233.1291369>
33. Campos, P.G., Díez, F., Cantador, I.: Time-aware recommender systems: a comprehensive survey and analysis of existing evaluation protocols. *User Model. User-Adapt. Interact.* **24**(1–2), 67–119 (2014)
34. Cantador, I., Castells, P.: Semantic contextualisation in a news recommender system. In: *Workshop on Context-Aware Recommender Systems (CARS 2009)*. New York (2009)
35. Cena, F., Console, L., Gena, C., Goy, A., Levi, G., Modeo, S., Torre, I.: Integrating heterogeneous adaptation techniques to build a flexible and usable mobile tourist guide. *AI Communications* **19**(4), 369–384 (2006)
36. Chatterjee, S., Hadi, A.S., Price, B.: *Regression analysis by example*. John Wiley and Sons (2000)
37. Chaudhuri, S., Dayal, U.: An overview of data warehousing and olap technology. *ACM Sigmod record* **26**(1), 65–74 (1997)

38. Cheverst, K., Davies, N., Mitchell, K., Friday, A., Efstratiou, C.: Developing a context-aware electronic tourist guide: some issues and experiences. In: Proceedings of the SIGCHI conference on Human factors in computing systems, pp. 17–24. ACM (2000)
39. Church, K., Smyth, B., Cotter, P., Bradley, K.: Mobile information access: A study of emerging search behavior on the mobile internet. *ACM Trans. Web* **1**(1) (2007). DOI [10.1145/1232722.1232726](https://doi.org/10.1145/1232722.1232726). URL <http://doi.acm.org/10.1145/1232722.1232726>
40. Codina, V., Ricci, F., Ceccaroni, L.: Exploiting the semantic similarity of contextual situations for pre-filtering recommendation. In: S. Carberry, S. Weibelzahl, A. Micarelli, G. Semeraro (eds.) *User Modeling, Adaptation, and Personalization, Lecture Notes in Computer Science*, vol. 7899, pp. 165–177. Springer Berlin Heidelberg (2013). DOI [10.1007/978-3-642-38844-6_14](https://doi.org/10.1007/978-3-642-38844-6_14). URL http://dx.doi.org/10.1007/978-3-642-38844-6_14
41. De Carolis, B., Mazzotta, I., Novielli, N., Silvestri, V.: Using common sense in providing personalized recommendations in the tourism domain. In: Workshop on Context-Aware Recommender Systems (CARS 2009). New York (2009)
42. Dourish, P.: What we talk about when we talk about context. *Personal and ubiquitous computing* **8**(1), 19–30 (2004)
43. Fling, B.: *Mobile Design and Development: Practical Concepts and Techniques for Creating Mobile Sites and Web Apps*, 1st edn. O'Reilly Media, Inc. (2009)
44. Gorgoglione, M., Panniello, U., Tuzhilin, A.: The effect of context-aware recommendations on customer purchasing behavior and trust. In: Proceedings of the Fifth ACM Conference on Recommender Systems, RecSys '11, pp. 85–92. ACM, New York, NY, USA (2011). DOI [10.1145/2043932.2043951](https://doi.org/10.1145/2043932.2043951). URL <http://doi.acm.org/10.1145/2043932.2043951>
45. Hariri, N., Mobasher, B., Burke, R.: Context-aware music recommendation based on latent topic sequential patterns. In: Proceedings of the Sixth ACM Conference on Recommender Systems, RecSys '12, pp. 131–138. ACM, New York, NY, USA (2012). DOI [10.1145/2365952.2365979](https://doi.org/10.1145/2365952.2365979). URL <http://doi.acm.org/10.1145/2365952.2365979>
46. Hariri, N., Mobasher, B., Burke, R.: Query-driven context aware recommendation. In: Proceedings of the 7th ACM Conference on Recommender Systems, RecSys '13, pp. 9–16. ACM, New York, NY, USA (2013). DOI [10.1145/2507157.2507187](https://doi.org/10.1145/2507157.2507187). URL <http://doi.acm.org/10.1145/2507157.2507187>
47. Hussein, T., Linder, T., Gaulke, W., Ziegler, J.: Context-aware recommendations on rails. In: Workshop on Context-Aware Recommender Systems (CARS 2009). New York, NY, USA (2009)
48. Hussein, T., Linder, T., Gaulke, W., Ziegler, J.: Hybreed: A software framework for developing context-aware hybrid recommender systems. *User Model. User-Adapt. Interact.* **24**(1–2), 121–174 (2014)
49. Jannach, D., Hegelich, K.: A case study on the effectiveness of recommendations in the mobile internet. In: Proceedings of the Third ACM Conference on Recommender Systems, RecSys '09, pp. 205–208. ACM, New York, NY, USA (2009). DOI [10.1145/1639714.1639749](https://doi.org/10.1145/1639714.1639749). URL <http://doi.acm.org/10.1145/1639714.1639749>
50. Jiang, T., Tuzhilin, A.: Improving personalization solutions through optimal segmentation of customer bases. *IEEE Transactions on Knowledge and Data Engineering* **21**(3), 305–320 (2009)
51. Kaminskas, M., Ricci, F.: Contextual music information retrieval and recommendation: State of the art and challenges. *Computer Science Review* **6**(2-3), 89–119 (2012)
52. Karatzoglou, A., Amatriain, X., Baltrunas, L., Oliver, N.: Multiverse recommendation: N-dimensional tensor factorization for context-aware collaborative filtering. In: Proceedings of the Fourth ACM Conference on Recommender Systems, RecSys '10, pp. 79–86. ACM, New York, NY, USA (2010). DOI [10.1145/1864708.1864727](https://doi.org/10.1145/1864708.1864727). URL <http://doi.acm.org/10.1145/1864708.1864727>
53. Karatzoglou, A., Baltrunas, L., Church, K., Böhmer, M.: Climbing the app wall: Enabling mobile app discovery through context-aware recommendations. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, CIKM '12, pp. 2527–2530. ACM, New York, NY, USA (2012). DOI [10.1145/2396761.2398683](https://doi.org/10.1145/2396761.2398683). URL <http://doi.acm.org/10.1145/2396761.2398683>

54. Kenteris, M., Gavalas, D., Economou, D.: Electronic mobile guides: a survey. *Personal and Ubiquitous Computing* **15**(1), 97–111 (2011)
55. Kimball, R., Ross, M.: *The data warehousing toolkit*. John Wiley & Sons, New York (1996)
56. Kiseleva, J., Thanh Lam, H., Pechenizkiy, M., Calders, T.: Discovering temporal hidden contexts in web sessions for user trail prediction. In: *Proceedings of the 22nd International Conference on World Wide Web Companion, WWW '13 Companion*, pp. 1067–1074. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, Switzerland (2013). URL <http://dl.acm.org/citation.cfm?id=2487788.2488120>
57. Koller, D., Sahami, M.: Toward optimal feature selection. In: *Proceedings of the 13th International Conference on Machine Learning*, pp. 284–292. Morgan Kaufmann (1996)
58. Koren, Y.: Factorization meets the neighborhood: a multifaceted collaborative filtering model. In: *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 426–434. ACM, New York, NY (2008)
59. Koutrika, G., Bercovitz, B., Garcia-Molina, H.: Flexrecs: expressing and combining flexible recommendations. In: *Proceedings of the 35th SIGMOD international conference on Management of data*, pp. 745–758. ACM, Providence, RI (2009)
60. Lee, H., Park, S.J.: Moners: A news recommender for the mobile web. *Expert Systems with Applications* **32**(1), 143–150 (2007). DOI <http://dx.doi.org/10.1016/j.eswa.2005.11.010>. URL <http://www.sciencedirect.com/science/article/pii/S0957417405003167>
61. Levandoski, J.J., Ekstrand, M.D., Ludwig, M., Eldawy, A., Mokbel, M.F., Riedl, J.: Recbench: Benchmarks for evaluating performance of recommender system architectures. *PVLDB* **4**(11), 911–920 (2011)
62. Levandoski, J.J., Eldawy, A., Mokbel, M.F., Khalefa, M.E.: Flexible and extensible preference evaluation in database systems. *ACM Trans. Database Syst.* **38**(3), 17 (2013)
63. Liu, H., Motoda, H.: *Feature selection for knowledge discovery and data mining*. Springer (1998)
64. Lombardi, S., Anand, S.S., Gorgoglione, M.: Context and customer behavior in recommendation. In: *Workshop on Context-Aware Recommender Systems (CARS 2009)*. New York
65. Mahmood, T., Ricci, F., Venturini, A.: Improving recommendation effectiveness: Adapting a dialogue strategy in online travel planning. *J. of IT & Tourism* **11**(4), 285–302 (2009). DOI [10.3727/109830510X12670455864203](https://doi.org/10.3727/109830510X12670455864203). URL <http://dx.doi.org/10.3727/109830510X12670455864203>
66. Moling, O., Baltrunas, L., Ricci, F.: Optimal radio channel recommendations with explicit and implicit feedback. In: P. Cunningham, N.J. Hurley, I. Guy, S.S. Anand (eds.) *Sixth ACM Conference on Recommender Systems, RecSys '12*, Dublin, Ireland, September 9–13, 2012, pp. 75–82. ACM (2012). DOI [10.1145/2365952.2365971](https://doi.org/10.1145/2365952.2365971). URL <http://doi.acm.org/10.1145/2365952.2365971>
67. Odic, A., Tkalcic, M., Tasic, J.F., Kosir, A.: Predicting and detecting the relevant contextual information in a movie-recommender system. *Interacting with Computers* **25**(1), 74–90 (2013). DOI [10.1093/iwc/iws003](https://doi.org/10.1093/iwc/iws003). URL <http://dx.doi.org/10.1093/iwc/iws003>
68. Oku, K., Nakajima, S., Miyazaki, J., Uemura, S.: Context-aware SVM for context-dependent information recommendation. In: *Proceedings of the 7th International Conference on Mobile Data Management*, p. 109 (2006)
69. Palmisano, C., Tuzhilin, A., Gorgoglione, M.: Using context to improve predictive modeling of customers in personalization applications. *IEEE Transactions on Knowledge and Data Engineering* **20**(11), 1535–1549 (2008)
70. Panniello, U., Tuzhilin, A., Gorgoglione, M.: Comparing context-aware recommender systems in terms of accuracy and diversity. *User Model. User-Adapt. Interact.* **24**(1-2), 35–65 (2014)
71. Panniello, U., Tuzhilin, A., Gorgoglione, M., Palmisano, C., Pedone, A.: Experimental comparison of pre-vs. post-filtering approaches in context-aware recommender systems. In: *Proceedings of the 3rd ACM conference on Recommender systems*, pp. 265–268. ACM (2009)
72. Park, H.S., Yoo, J.O., Cho, S.B.: A context-aware music recommendation system using fuzzy bayesian networks with utility theory. In: *Proceedings of the Third International Conference on Fuzzy Systems and Knowledge Discovery, FSKD'06*, pp. 970–979. Springer-Verlag, Berlin, Heidelberg (2006). DOI [10.1007/11881599_121](https://doi.org/10.1007/11881599_121). URL http://dx.doi.org/10.1007/11881599_121

73. Park, M.H., Hong, J.H., Cho, S.B.: Location-based recommendation system using bayesian user's preference model in mobile devices. In: Proceedings of the 4th International Conference on Ubiquitous Intelligence and Computing, UIC'07, pp. 1130–1139. Springer-Verlag, Berlin, Heidelberg (2007). URL <http://dl.acm.org/citation.cfm?id=2391319.2391438>
74. Pennock, D.M., Horvitz, E.: Collaborative filtering by personality diagnosis: A hybrid memory-and model-based approach. In: IJCAI'99 Workshop: Machine Learning for Information Filtering (1999)
75. Ramakrishnan, R., Gehrke, J.: Database Management Systems. USA: McGraw Hill Companies (2000)
76. Reddy, S., Mascia, J.: Lifetrak: Music in tune with your life. In: Proceedings of the 1st ACM International Workshop on Human-centered Multimedia, HCM '06, pp. 25–34. ACM, New York, NY, USA (2006). DOI [10.1145/1178745.1178754](https://doi.org/10.1145/1178745.1178754). URL <http://doi.acm.org/10.1145/1178745.1178754>
77. Ricci, F.: Mobile recommender systems. J. of IT & Tourism **12**(3), 205–231 (2010)
78. Ricci, F., Nguyen, Q.N.: Mobyrek: A conversational recommender system for on-the-move travelers. Destination Recommendation Systems: Behavioural Foundations and Applications pp. 281–294 (2006)
79. Ricci, F., Rokach, L., Shapira, B., Kantor, P.B. (eds.): Recommender Systems Handbook. Springer (2011)
80. Sae-Ueng, S., Pinyapong, S., Ogino, A., Kato, T.: Personalized shopping assistance service at ubiquitous shop space. In: Proceedings of the 22nd International Conference on Advanced Information Networking and Applications - Workshops, AINAW '08, pp. 838–843. IEEE Computer Society, Washington, DC, USA (2008). DOI [10.1109/WAINA.2008.287](https://doi.org/10.1109/WAINA.2008.287). URL <http://dx.doi.org/10.1109/WAINA.2008.287>
81. Sarwar, B., Karypis, G., Konstan, J., Reidl, J.: Item-based collaborative filtering recommendation algorithms. In: Proceedings of the 10th international conference on World Wide Web, pp. 285–295. ACM (2001)
82. Sarwat, M., Avery, J., Mokbel, M.F.: A recdb in action: Recommendation made easy in relational databases. PVLDB **6**(12), 1242–1245 (2013)
83. Sarwat, M., Levandoski, J.J., Eldawy, A., Mokbel, M.F.: Lars*: An efficient and scalable location-aware recommender system. IEEE Trans. Knowl. Data Eng. **26**(6), 1384–1399 (2014)
84. Shani, G., Gunawardana, A.: Evaluating recommendation systems. In: Ricci et al. [79], pp. 257–297
85. Smyth, B., Cotter, P.: Mp3 mobile portals, profiles and personalization. In: Web Dynamics, pp. 411–433. Springer Berlin Heidelberg (2004). DOI [10.1007/978-3-662-10874-1_17](https://doi.org/10.1007/978-3-662-10874-1_17). URL http://dx.doi.org/10.1007/978-3-662-10874-1_17
86. Van Setten, M., Pokraev, S., Koolwaaij, J.: Context-aware recommendations in the mobile tourist application COMPASS. In: W. Nejdl, P. De Bra (eds.) Adaptive Hypermedia, pp. 235–244. Springer Verlag (2004)
87. Webster, N.: Webster's new twentieth century dictionary of the English language. Springfield, MA: Merriam-Webster, Inc. (1980)
88. Woerndl, W., Huebner, J., Bader, R., Gallego-Vico, D.: A model for proactivity in mobile, context-aware recommender systems. In: Proceedings of the Fifth ACM Conference on Recommender Systems, RecSys '11, pp. 273–276. ACM, New York, NY, USA (2011). DOI [10.1145/2043932.2043981](https://doi.org/10.1145/2043932.2043981). URL <http://doi.acm.org/10.1145/2043932.2043981>
89. Yu, Z., Zhou, X., Zhang, D., Chin, C.Y., Wang, X., Men, J.: Supporting context-aware media recommendations for smart phones. IEEE Pervasive Computing **5**(3), 68–75 (2006)
90. Zheng, Y., Burke, R., Mobasher, B.: Differential context relaxation for context-aware travel recommendation. In: C. Huemer, P. Lops (eds.) E-Commerce and Web Technologies, *Lecture Notes in Business Information Processing*, vol. 123, pp. 88–99. Springer Berlin Heidelberg (2012). DOI [10.1007/978-3-642-32273-0_8](https://doi.org/10.1007/978-3-642-32273-0_8). URL http://dx.doi.org/10.1007/978-3-642-32273-0_8